Week 6 Asynchronous Materials Guide - Lists and Tuples

This Week's Assignment: Pseudocode Project

Instead of regular homework, you'll design an algorithm using pseudocode and the materials below. After you finish the asynchronous guide look at the homework assignment for more details

Start Here: Video Tutorial

Corey Schafer Python Lists Tutorial

Python Tutorial: Lists, Sets, and Tuples

- **Duration:** Watch 0:00 - 205:00 (25 minutes - just the list and tuple sections)
- **What you will learn:** List creation, indexing, slicing, and common methods
- **Why this helps:** Reinforces lecture concepts with different examples
- **Study approach:** Pause and try each example in Kaggle yourself

Creating Lists - Beyond the Basics

Monday's lecture covered basic list creation. These materials add more techniques.

W3Schools Python Lists

https://www.w3schools.com/python/python_lists.asp

- Review basic list syntax
- Try the interactive "Try it Yourself" examples
- Test different list operations

Advanced List Creation Techniques

```
# Creating from strings
sentence = "Python is amazing"
words = sentence.split()  # ["Python", "is", "amazing"]
print(f"Words: {words}")

# Split with different separator
csv_data = "apple,banana,cherry"
fruits = csv_data.split(",")  # ["apple", "banana", "cherry"]
print(f"Fruits: {fruits}")

# Creating from range
numbers = list(range(5))  # [0, 1, 2, 3, 4]
print(f"Range: {numbers}")

countdown = list(range(10, 0, -1))  # [10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
print(f"Countdown: {countdown}")

evens = list(range(0, 11, 2))  # [0, 2, 4, 6, 8, 10]
print(f"Evens: {evens}")

# Repetition with multiplication
zeros = [0] * 5  # [0, 0, 0, 0, 0]
print(f"Zeros: {zeros}")

pattern = ["A", "B"] * 3  # ["A", "B", "A", "B", "A", "B"]
print(f"Pattern: {pattern}")

dashes = ["-"] * 10  # ["-", "-", "-", "-", "-", "-", "-", "-", "-", "-"]
separator = "".join(dashes)  # Convert list to string
print(f"Separator: {separator}")

# Lists can hold different types
mixed = [1, "hello", 3.14, True, [1, 2, 3]]
print(f"Mixed types: {mixed}")

# Nested lists (lists inside lists)
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
print(f"Matrix: {matrix}")
```

```
print(f"First row: {matrix[0]}")
print(f"Middle element: {matrix[1][1]}")  # 5
```

Indexing - Edge Cases and Best Practices

Monday's lecture covered positive and negative indexing. These materials add safe indexing practices.

Real Python - List Indexing

https://realpython.com/python-lists-tuples/#list-elements-can-be-accessed-by-index

- Deep dive into indexing mechanics
- Understanding IndexError
- Safe indexing practices

Safe Indexing Practices

```
# Always check length before indexing
scores = [85, 92, 78]
print(f"Length: {len(scores)}")

# Safe way to access last element
if len(scores) > 0:
    last_score = scores[-1]
    print(f"Last score: {last_score}")
else:
    print("List is empty!")

# Check if index exists
index = 5
if index < len(scores):
    value = scores[index]
    print(f"Value at {index}: {value}")
else:
    print(f"Index {index} doesn't exist!")

# Getting last index
last_index = len(scores) - 1  # 2
print(f"Last valid index: {last_index}")
```

```
# Modifying elements safely
if len(scores) >= 2:
    scores[1] = 95  # Change second score
    print(f"Updated scores: {scores}")

# Using max/min with indices
numbers = [10, 25, 5, 30, 15]
max_value = max(numbers)
min_value = min(numbers)
max_index = numbers.index(max_value)
min_index = numbers.index(min_value)

print(f"Max: {max_value} at index {max_index}")
print(f"Min: {min_value} at index {min_index}")
```

Slicing - The Step Parameter

> Monday's lecture covered basic slicing with start and end. These materials add the step parameter.

Python Slicing Tutorial

https://www.programiz.com/python-programming/methods/list/slice

- Complete slicing reference
- Step parameter explained
- Advanced slicing patterns

Advanced Slicing with Step

```
# Working with a list
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
print(f"Original: {numbers}")
print()

# Using step to skip elements
every_second = numbers[::2]  # [0, 2, 4, 6, 8]
print(f"Every second [::2]: {every_second}")

every_third = numbers[::3]   # [0, 3, 6, 9]
print(f"Every third [::3]: {every_third}")
```

```python
# Start at different position with step
odds = numbers[1::2]  # [1, 3, 5, 7, 9]
print(f"Odds [1::2]: {odds}")
print()

# Combining start, end, and step
subset = numbers[2:8:2]  # [2, 4, 6]
print(f"From 2 to 8, every 2nd [2:8:2]: {subset}")
print()

# REVERSING with negative step
reversed_all = numbers[::-1]  # [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
print(f"Reversed [::-1]: {reversed_all}")

reversed_evens = numbers[::-2]  # [9, 7, 5, 3, 1]
print(f"Reversed, every 2nd [::-2]: {reversed_evens}")
print()

# Practical string reversal
word = "Python"
letters = list(word)  # ["P", "y", "t", "h", "o", "n"]
reversed_letters = letters[::-1]  # ["n", "o", "h", "t", "y", "P"]
reversed_word = "".join(reversed_letters)  # "nohtyP"
print(f"{word} reversed: {reversed_word}")
print()

# Checking palindromes
def is_palindrome(text):
    letters = list(text.lower())
    return letters == letters[::-1]

print(f"'racecar' is palindrome: {is_palindrome('racecar')}")
print(f"'python' is palindrome: {is_palindrome('python')}")
```

Practical Slicing Patterns

```python
# Get first N items
top_5 = [100, 95, 90, 88, 85, 82, 80][:5]
print(f"Top 5: {top_5}")

# Get last N items
```

```
last_3 = [1, 2, 3, 4, 5, 6, 7, 8][-3:]
print(f"Last 3: {last_3}")

# Remove first and last
middle_only = [10, 20, 30, 40, 50][1:-1]
print(f"Middle only: {middle_only}")

# Copy a list
original = [1, 2, 3]
copy = original[:]
copy[0] = 999
print(f"Original: {original}")  # Unchanged!
print(f"Copy: {copy}")
```

List Methods - Beyond the Basics

Monday's lecture covered `.append()`, `.pop()`, `.remove()`, and `len()`. These materials add more methods.

W3Schools List Methods Reference

https://www.w3schools.com/python/python_lists_methods.asp

- Complete list of all methods
- Examples for each
- Try them interactively

Additional List Methods

```
# INSERT at specific position
fruits = ["apple", "cherry"]
print(f"Before insert: {fruits}")

fruits.insert(1, "banana")  # Insert at index 1
print(f"After insert(1, 'banana'): {fruits}")

fruits.insert(0, "apricot")  # Insert at beginning
print(f"After insert(0, 'apricot'): {fruits}")
print()
```

```python
# EXTEND - add multiple items
numbers = [1, 2, 3]
print(f"Before extend: {numbers}")

numbers.extend([4, 5, 6])
print(f"After extend([4,5,6]): {numbers}")

# Difference between extend and append
list1 = [1, 2]
list1.append([3, 4])  # Adds the list as ONE item
print(f"After append([3,4]): {list1}")  # [1, 2, [3, 4]]

list2 = [1, 2]
list2.extend([3, 4])  # Adds items individually
print(f"After extend([3,4]): {list2}")  # [1, 2, 3, 4]
print()

# INDEX - find position of item
colors = ["red", "blue", "green", "blue", "yellow"]
position = colors.index("blue")  # Finds FIRST occurrence
print(f"First 'blue' at index: {position}")

# Check if item exists before using index
if "purple" in colors:
    pos = colors.index("purple")
else:
    print("'purple' not in list")
print()

# COUNT - how many times item appears
numbers = [1, 2, 3, 2, 4, 2, 5]
count = numbers.count(2)
print(f"Number 2 appears {count} times")
print()

# CLEAR - remove all items
temp = [1, 2, 3, 4, 5]
print(f"Before clear: {temp}")
temp.clear()
print(f"After clear: {temp}")
print()

# REVERSE - flip the order
letters = ["A", "B", "C", "D"]
```

```
print(f"Before reverse: {letters}")
letters.reverse()
print(f"After reverse: {letters}")
print()

# SORT - organize items
scores = [85, 92, 78, 95, 88]
print(f"Before sort: {scores}")

scores.sort()
print(f"After sort: {scores}")

scores.sort(reverse=True)
print(f"Descending: {scores}")

# Sorting strings (alphabetical)
names = ["Charlie", "Alice", "Bob"]
names.sort()
print(f"Sorted names: {names}")
```

Critical Difference: Modify vs Return

Some methods **modify the list in place** and return `None`:

- `.append()`, `.insert()`, `.extend()`, `.remove()`, `.pop()`
- `.sort()`, `.reverse()`, `.clear()`

Others **return a new value** without changing the original:

- `sorted(list)` - returns new sorted list
- `len(list)`, `.index()`, `.count()` - return information

```
# WRONG - sort() returns None!
numbers = [3, 1, 4]
sorted_numbers = numbers.sort()
print(sorted_numbers)  # None!

# CORRECT approach 1: sort in place
numbers = [3, 1, 4]
numbers.sort()
print(numbers)  # [1, 3, 4]

# CORRECT approach 2: create sorted copy
```

```
numbers = [3, 1, 4]
sorted_numbers = sorted(numbers)
print(f"Original: {numbers}")  # [3, 1, 4]
print(f"Sorted: {sorted_numbers}")  # [1, 3, 4]
```

Tuples

Monday's lecture covered tuple basics: use parentheses `()`, cannot be modified. These materials add unpacking and use cases.

Real Python Tuples Guide

https://realpython.com/python-lists-tuples/#python-tuples

- Comprehensive tuple tutorial
- Tuple packing and unpacking
- When to choose tuples over lists

Working with Tuples

```
# Creating tuples
empty_tuple = ()
single_item = (42,)  # Note comma! Without it (42) is just an int
coordinates = (10, 20)
rgb_color = (255, 128, 0)
person = ("Alice", 25, "Engineer")

print(f"Person: {person}")
print()

# Tuple packing (no parentheses needed)
point = 5, 10  # Same as (5, 10)
print(f"Point: {point}, Type: {type(point)}")
print()

# Indexing and slicing (same as lists!)
name = person[0]      # "Alice"
age = person[1]       # 25
```

```python
first_two = person[:2]  # ("Alice", 25)

print(f"Name: {name}, Age: {age}")
print(f"First two: {first_two}")
print()

# TUPLE UNPACKING - very powerful!
x, y = coordinates
print(f"x = {x}, y = {y}")

name, age, job = person
print(f"{name} is {age} and works as a {job}")
print()

# Swapping values with tuple unpacking
a = 5
b = 10
print(f"Before swap: a={a}, b={b}")

a, b = b, a  # Swap!
print(f"After swap: a={a}, b={b}")
print()

# Tuples are IMMUTABLE
try:
    coordinates[0] = 15  # This will error!
except TypeError as e:
    print(f"Error: {e}")
    print("Cannot modify tuples!")
print()

# What tuples CAN do
print(f"Length: {len(person)}")
print(f"Count 'Alice': {person.count('Alice')}")
print(f"Index of 25: {person.index(25)}")
print(f"25 in tuple: {25 in person}")
print()

# Returning multiple values from functions
def get_user_stats():
    username = "alice123"
    score = 9500
    level = 15
    return username, score, level  # Returns a tuple!
```

```
user, points, lvl = get_user_stats()
print(f"User: {user}, Points: {points}, Level: {lvl}")
```

Lists vs Tuples - Complete Comparison

Feature	Lists `[]`	Tuples `()`
Mutable	✅ Can change after creation	❌ Cannot change (immutable)
Syntax	Square brackets: `[1, 2, 3]`	Parentheses: `(1, 2, 3)`
Methods	Many: append, insert, remove, pop, sort, reverse, extend, clear	Only two: count, index
Speed	Slightly slower	Slightly faster
Memory	Uses more memory	Uses less memory
Use Cases	Shopping carts, game scores, user inputs - data that changes	Coordinates, RGB values, database records - fixed data

Decision Guide: Which Should I Use?

Use Lists when:

- You need to add, remove, or modify elements
- The data will change over time
- Order might need to change (sorting)
- Examples: shopping cart items, player inventory, test scores, to-do list

Use Tuples when:

- Data should never change (x,y coordinates, RGB colors)
- Returning multiple values from a function
- You want to protect data from accidental modification

- Examples: (latitude, longitude), (year, month, day), database row, configuration settings

Common Mistakes to Avoid

Mistake 1: Forgetting End Index is Excluded

```
numbers = [0, 1, 2, 3, 4]
subset = numbers[1:3]  # Gets indices 1 and 2, NOT 3!
print(subset)  # [1, 2]

# If you want indices 1, 2, 3:
subset = numbers[1:4]  # End is 4, not 3!
print(subset)  # [1, 2, 3]
```

Mistake 2: Assigning sort() Result

```
# WRONG
numbers = [3, 1, 4]
sorted_numbers = numbers.sort()
print(sorted_numbers)  # None!

# CORRECT
numbers.sort()  # Modifies in place
print(numbers)  # [1, 3, 4]

# OR use sorted()
numbers = [3, 1, 4]
sorted_numbers = sorted(numbers)
print(sorted_numbers)  # [1, 3, 4]
```

Mistake 3: Modifying While Checking

```python
# Be careful when index might not exist
colors = ["red", "blue"]

# UNSAFE
# color = colors[5]  # IndexError!

# SAFE
if len(colors) > 5:
    color = colors[5]
else:
    print("Index doesn't exist")

# Or use get-like pattern
index = 5
color = colors[index] if index < len(colors) else "Not found"
print(color)
```

LearnPython.org Interactive Practice

https://www.learnpython.org/en/Lists

- Interactive exercises
- Immediate feedback
- Test your understanding

Quick Reference Cheat Sheet

List Operations Summary

```python
# Creating
my_list = [1, 2, 3]
my_list = list(range(5))  # [0, 1, 2, 3, 4]
my_list = [0] * 5  # [0, 0, 0, 0, 0]

# Accessing
first = my_list[0]
last = my_list[-1]
subset = my_list[1:3]
```

```
reversed = my_list[::-1]

# Modifying
my_list[0] = 10           # Change element
my_list.append(4)         # Add to end
my_list.insert(0, 0)      # Insert at position
my_list.extend([5, 6])    # Add multiple

# Removing
my_list.remove(10)        # Remove first occurrence
item = my_list.pop()      # Remove and return last
item = my_list.pop(0)     # Remove and return at index

# Searching
index = my_list.index(3)  # Find position
count = my_list.count(2)  # Count occurrences
exists = 3 in my_list     # Check existence

# Organizing
my_list.sort()            # Sort in place
my_list.reverse()         # Reverse in place
sorted_copy = sorted(my_list)  # Return sorted copy

# Info
length = len(my_list)
maximum = max(my_list)
minimum = min(my_list)
total = sum(my_list)
```

Tuple Operations Summary

```
# Creating
my_tuple = (1, 2, 3)
my_tuple = 1, 2, 3        # Tuple packing
single = (42,)            # Single item (note comma!)

# Accessing (same as lists)
first = my_tuple[0]
last = my_tuple[-1]

# Unpacking
a, b, c = my_tuple
x, y = (10, 20)
```

```python
# Operations
length = len(my_tuple)
count = my_tuple.count(2)
index = my_tuple.index(3)
exists = 2 in my_tuple

# CANNOT modify!
# my_tuple[0] = 10        # ERROR
# my_tuple.append(4)      # ERROR
```